INSTITUTO TÉCNICO COMERCIAL DE LUANDA

DESENVOLVIMENTO DE UM SISTEMA DE RECUPERAÇÃO DE DOCUMENTOS PERDIDOS BASEADO EM OCR

ESTUDO DE CASO: ANGOLA

Nome do Aluno: Casimiro Luzolo Gaspar Puaia

Nº: 5

Turma: DTI

Classe: 13ª

Sala: 16

Luanda, 2026/2027

ÍNDICE

1. RESUMO **3**
2. PROBLEMÁTICA **3**
3. PERGUNTA DE PESQUISA **4**
4. HIPÓTESES **4**
5. OBJETIVOS **4**
6.1 Objetivo geral 4
6.2 Objetivos específicos 4
6. JUSTIFICATIVA **5**
7. ESTUDO COMPARATIVO DAS TECNOLOGIAS **5**
7.1 Motores de OCR: Tesseract, EasyOCR, PaddleOCR e serviços em nuvem 5
7.2 Motores de indexação e busca: Elasticsearch ou PostgreSQL? 6
7.3 Linguagem do servidor: Python ou JavaScript (Node.js)? 7
7.4 Bases de dados relacionais: PostgreSQL ou MySQL? 8
7.5 Conclusão do estudo comparativo 9
8. TECNOLOGIAS UTILIZADAS **9**
8.1 Arquitetura geral 9
8.2 Frontend: HTML5, CSS3, JavaScript (React) 10
8.3 Backend: Python e FastAPI 10
8.4 Pré-processamento de imagem: OpenCV e Pillow 10
8.5 Reconhecimento ótico de caracteres: Tesseract e PaddleOCR 10
8.6 Base de dados e índices: PostgreSQL, pg_trgm e pgvector 11
8.7 Segurança, privacidade e conformidade legal 11
8.8 Contentorização e implantação: Docker, Nginx e CI 12
8.9 Limitações e trabalho futuro 12
9. CONCLUSÃO **13**
10. REFERÊNCIAS BIBLIOGRAFICAS **14**

1. RESUMO

O presente trabalho consiste em criar um sistema web que permita a digitalização, ou a inserção textual, de dados de documentos perdidos e encontrados. O objetivo é o cadastro de documentos extraviados e a busca eficiente de documentos físicos ou digitais, combinando técnicas de OCR (*Optical Character Recognition*, Reconhecimento Ótico de Caracteres) e de recuperação de informação (*Information Retrieval*, IR).

A avaliação do sistema usará métricas de precisão e de revocação (*recall*) para medir a eficácia do cadastro e da recuperação de documentos. A combinação de OCR de alta qualidade com métodos de Processamento de Linguagem Natural (NLP) e Inteligência Artificial, como os *embeddings* vetoriais, tende a maximizar o *recall*. Já os métodos tradicionais baseados em palavras-chave favorecem a precisão. Recomenda-se, por isso, pesquisa adicional em técnicas híbridas e testes práticos com conjuntos de dados reais.

2. PROBLEMÁTICA

Nota-se que recuperação de um documento perdido depende hoje, em grande medida, de canais informais e não integrados (normalmente se vê documentos pendurados em janelas de taxis e até mesmo em lojas). Quem perde um documento não sabe onde procurar. Quem o encontra não sabe a quem entregá-lo, nem se o titular já o procurou. Mesmo quando há registos, estes são feitos em papel, em livros de ocorrências ou em mensagens soltas de redes sociais, sem estrutura e sem possibilidade de busca eficiente.

Mesmo num sistema digital, a correspondência entre o registo do achado e o registo da perda é difícil por quatro razões:

- **Dados incompletos.** O utilizador que perdeu o documento pode recordar apenas o nome, ou apenas parte do número.
- **Erros de transcrição e de OCR.** O reconhecimento ótico de documentos com fundos complexos, selos e fotografias produz erros de caracteres (por exemplo, confusão entre 0 e O, ou entre 1 e I), que impedem a correspondência exata.
- **Variação linguística.** Os nomes têm grafias alternativas, abreviaturas e ordem diferente dos apelidos, o que degrada a busca por palavras-chave exatas.
- **Descrições em texto livre.** Muitos registos têm descrições informais (“carteira preta com BI e cartão do banco”).

Existe também um risco de privacidade. Um sistema público que exponha dados completos de documentos achados pode facilitar a burla e o uso indevido. O problema, então, é duplo: construir um mecanismo de cadastro e busca capaz de reunir documentos achados e documentos procurados com alto *recall* e boa precisão, e fazê-lo sem expor desnecessariamente dados pessoais.

3. PERGUNTA DE PESQUISA

Pergunta principal: Em que medida um sistema web que combina OCR com métodos de recuperação de informação lexicais e semânticos melhora a eficácia da recuperação de documentos perdidos em Angola?

4. HIPÓTESES

- **H1.** Ao criar uma plataforma destinada apenas na recuperação de documentos perdidos poderá melhorar significativamente a taxa de recuperabilidade.
- **H2.** Cidadãos que perderam ou acharam documentos teriam um ponto de primeiro destino para a procura
- **H3.** Reduz a exposição dos dados de documentos achados e colocados a exposição em locais públicos

Perguntas secundárias:

- Que motor de OCR oferece a melhor relação entre exatidão, custo e viabilidade de implantação para documentos em português do contexto angolano?
- Qual o melhor conjunto de tecnologias atende as necessidades do projeto?
- Como equilibrar a eficácia da busca com a proteção de dados pessoais?

Respostas:

- **R1.** O Tesseract, apoiado por pré-processamento de imagem com OpenCV, oferece o melhor equilíbrio entre exatidão, custo e viabilidade de implantação, com o PaddleOCR como motor de reserva para imagens difíceis..
- **R2.** O conjunto formado por Python (FastAPI), React e PostgreSQL com pg_trgm e pgvector, porque reúne OCR, busca lexical, aproximada e vetorial num só serviço, em código aberto e de baixo custo.
- **R3.** Minimizando os dados mostrados (só o início e o fim do número e o nome parcialmente ocultado), cifrando os dados e verificando o dono antes de entregar o contacto.

5. OBJETIVOS

6.1 Objetivo geral

Desenvolver um sistema web de recuperação de documentos perdidos, baseado em OCR e em recuperação de informação híbrida (lexical e semântica).

6.2 Objetivos específicos

- Caracterizar o problema da perda e do achado de documentos, identificando tipos de documentos e fluxos atuais.
- Comparar tecnologias candidatas (motores de OCR, linguagens de programação, bases de dados e abordagens de busca) e justificar a seleção do conjunto tecnológico.
- Implementar um módulo de busca com índice invertido, correspondência aproximada de nomes e busca vetorial, bem como a fusão híbrida dos resultados.

6. JUSTIFICATIVA

Em Angola, o cidadão depende de um conjunto de documentos para aceder a serviços públicos e privados: o Bilhete de Identidade (BI), o passaporte, a carta de condução, o livrete e título de propriedade de viaturas, o cartão de contribuinte e até os cartões multicaixas. A perda ou o extravio de qualquer um deles obriga qualquer um a uma sequência de passos que podemos dizer complexos onde inclui a participação do extravio e o pedido de segunda via. Nesse cenário, o desenvolvimento e implementação do sistema torna-se uma prática essencial e "indispensavel", especialmente após o aumento do acesso a internet e as redes sociais comparados as gerações anteriores

7. ESTUDO COMPARATIVO DAS TECNOLOGIAS

Aqui iremos comparar as alternativas tecnológicas encontradas durante a pesquisa para o desenvolvimento do sistema: motor de OCR, abordagem de recuperação de informação, motor de indexação e busca, linguagem de programação do servidor e sistema de gestão de base de dados. Em cada comparação iremos terminar com a escolha e a justificativa. A comparação baseia-se no conhecimento prévio sobre cada tecnologia, tempo de aprendizado (no caso de não saber nada), as suas areas de atuação e o custo de uso.

7.1 Motores de OCR: Tesseract, EasyOCR, PaddleOCR e serviços em nuvem

O OCR converte a imagem do documento em texto pesquisável. O **Tesseract** é um motor de código aberto, originalmente desenvolvido pela Hewlett-Packard e mantido por uma comunidade ativa. Tem modelos para português e funciona offline, em CPU. O seu ponto fraco é a sensibilidade à qualidade da imagem. Fundos texturados, selos sobrepostos ao texto, baixa resolução e distorção de perspetiva reduzem muito a exatidão, pelo que depende de bom pré-processamento.

O **EasyOCR** e o **PaddleOCR** são motores de aprendizagem profunda. Separam a deteção de texto, que localiza as regiões com texto, do reconhecimento, que lê o conteúdo de cada região. O PaddleOCR, em particular, foi desenhado para ser leve e prático (Du et al., 2020). Em imagens complexas, como fotografias de documentos tiradas com telemóvel, tendem a ser mais robustos do que o Tesseract, à custa de maior consumo de memória e de dependências mais pesadas. Modelos baseados em *transformers*, como o TrOCR, elevam ainda mais a exatidão em texto manuscrito e impresso, mas exigem GPU para uma inferência com latência aceitável (Li et al., 2023).

Os **serviços em nuvem**, como o Google Cloud Vision, oferecem OCR de alta exatidão sem gestão de infraestrutura. Em contrapartida, enviam imagens de documentos de identidade para servidores fora de Angola. O que levanta questões de exposição internacional de dados pessoais além de gerar custo por página e dependência de ligação à Internet.

Tabela 1. Comparação de motores de OCR

Critério	Tesseract	EasyOCR	PaddleOCR	Nuvem (ex.: Cloud Vision)
Licença	Apache 2.0, código aberto			Proprietária, pago por uso
Abordagem	LSTM sobre linhas de texto	Deteção + reconhecimento	Deteção + classificação + reconhecimento	Modelos proprietários, via API
Suporte ao português	Sim			
Robustez em fotos de telemóvel	Média; depende de pré-processamento	Boa	Boa a muito boa	Muito boa
Recursos computacionais	Baixos (CPU)	Médios (CPU ou GPU)	Médios (CPU ou GPU)	Nenhuns no cliente
Funciona offline	Sim			Não
Privacidade dos dados	Total (local)			Dados saem do país
Custo	Gratuito			Por página/imagem

Escolha. O Tesseract é o motor principal, por ser leve, gratuito, local, adequado a servidores modestos e por ser simples para aprendizado. É precedido de um pré-processamento com OpenCV. O PaddleOCR fica previsto como motor de contingência, acionado quando a confiança média do Tesseract fica abaixo. Os serviços em nuvem ficam excluídos por razões de privacidade e de custo.

7.2 Motores de indexação e busca: Elasticsearch ou PostgreSQL?

O **Elasticsearch** é um motor distribuído de busca baseado na biblioteca Lucene. Oferece índice invertido, BM25, analisadores por língua, agregações e, nas versões recentes, busca vetorial (Gormley & Tong, 2015). É a opção clássica quando há grandes volumes de dados e necessidade de escala horizontal. O custo é operacional: consome muita memória, exige configuração e manutenção, e acrescenta um segundo sistema que tem de ser sincronizado com a base de dados.

O **PostgreSQL** oferece pesquisa de texto integral nativa, com o tipo *tsvector*, índices GIN e configuração para português (PostgreSQL Global Development Group, 2024). A extensão *pg_trgm* acrescenta similaridade por trigramas, útil para nomes com erros de grafia. A extensão *pgvector* acrescenta vetores e índices de vizinhos mais próximos aproximados, como o HNSW, descrito por Malkov e Yashunin (2020) (Kane, s.d.). Assim, uma única base de dados serve dados relacionais, busca lexical, busca aproximada e busca vetorial, com transações e consistência.

Tabela 2. Alternativas de indexação e busca

Critério	Elasticsearch	PostgreSQL + pg_trgm + pgvector
Busca lexical (BM25)	Nativa, muito completa	Texto integral nativo (ts_rank); BM25 por extensão
Correspondência aproximada	Sim (fuzziness, n-gramas)	Sim (pg_trgm)
Busca vetorial	Sim (versões recentes)	Sim (pgvector, HNSW/IVFFlat)
Escala	Horizontal, muito grande	Vertical, adequada a milhões de registos
Complexidade operacional	Alta (cluster extra)	Baixa (um só serviço)
Consistência transacional	Eventual (sincronização)	ACID
Custo de infraestrutura	Alto (memória)	Baixo

Escolha. O PostgreSQL com *pg_trgm* e *pgvector* é a opção para o piloto. Reduz a complexidade, mantém a consistência entre metadados, texto e vetores e é compatível com servidores de baixo custo.

7.3 Linguagem do servidor: Python ou JavaScript (Node.js)?

O servidor tem de coordenar o recebimento de imagens, o pré-processamento, o OCR, a geração de *embeddings* e a busca. O **Python** é a linguagem dominante em ciência de dados e IA. Tem bibliotecas maduras para o pipeline do projeto. O *framework* FastAPI permite criar APIs REST assíncronas com validação automática e documentação OpenAPI.

O **JavaScript**, com Node.js, destaca-se pelo modelo de I/O, que o torna adequado a muitas ligações simultâneas, e pela vantagem de usar a mesma linguagem no *frontend* e no *backend*. Mas o seu ecossistema de IA é menos completo. Existem ligações para Tesseract (Tesseract.js), mas com menor desempenho e menos opções. Na prática, um servidor Node.js teria de chamar serviços em Python para a parte de OCR e de NLP.

Tabela 3. Python versus JavaScript (Node.js) no servidor

Critério	Python (FastAPI)	JavaScript (Node.js/Express)
Ecossistema de IA, OCR e NLP	Muito amplo e maduro	Limitado; frequentemente delega em Python
Processamento de imagem	OpenCV, Pillow, scikit-image	sharp, OpenCV.js (menos completo)
Desempenho em I/O concorrente	Bom (assíncrono com ASGI)	Muito bom (event loop)
Desempenho em tarefas de CPU	Bom com bibliotecas nativas (C/C++)	Limitado pelo modelo mono-thread
Tipagem e validação	Pydantic e type hints	TypeScript (opcional)
Unificação com o frontend	Não (duas linguagens)	Sim (uma linguagem)
Curva de aprendizagem	Suave	Suave, mas com muitas escolhas de bibliotecas

Escolha. O Python com FastAPI é a escolha para o *backend*, porque o núcleo do projeto, OCR e recuperação semântica, está no ecossistema Python. O JavaScript é usado no *frontend*, onde é a escolha natural para a interface web e para a captura de imagem no navegador.

7.4 Bases de dados relacionais: PostgreSQL ou MySQL?

O **MySQL** é um dos sistemas mais difundidos, com desempenho muito bom em leituras simples e grande disponibilidade em alojamento partilhado. A pesquisa de texto integral existe (índices FULLTEXT), mas é menos flexível do que a do PostgreSQL para configurações por língua. Não há, na distribuição comunitária, um equivalente maduro de *pgvector* para busca vetorial, o que obrigaria a usar um componente externo.

O **PostgreSQL** tem suporte mais completo ao padrão SQL, tipos avançados (JSONB, *arrays*, vetores), múltiplos tipos de índice e extensões. Estas características permitem concentrar nele texto, vetores e metadados (PostgreSQL Global Development Group, 2024).

Tabela 4. PostgreSQL versus MySQL

Critério	PostgreSQL	MySQL
Texto integral	tsvector, GIN, configuração em português	Índices FULLTEXT, menos configuráveis
Similaridade por n-gramas	pg_trgm	Não nativa
Busca vetorial	pgvector (HNSW, IVFFlat)	Sem extensão equivalente madura na versão comunitária
Tipos de dados avançados	JSONB, arrays, geometria, intervalos	JSON, tipos mais limitados
Extensibilidade	Muito alta	Moderada
Disponibilidade em alojamento barato	Boa e crescente	Muito alta
Conformidade ACID e SQL	Muito completa	Boa (InnoDB)

Escolha. Apesar do maior dominio e tempo de uso em MySQL, o PostgreSQL é a base de dados do sistema, por reunir num só serviço os requisitos de busca lexical, aproximada e vetorial.

7.5 Conclusão do estudo comparativo

A Tabela 6 resume as decisões tomadas. O critério dominante foi a adequação ao contexto angolano: privacidade e conformidade legal, baixo custo operacional, funcionamento em infraestrutura modesta e capacidade de crescer. Em todas as camadas se privilegiou o código aberto, o que reduz custos de licenciamento e facilita a auditoria.

Tabela 5. Decisões das tecnologias

Camada	Escolha	Justificação principal
OCR	**Tesseract (+ PaddleOCR de contingência)**	**Local, gratuito e leve; contingência para imagens difíceis**
Recuperação	**Híbrida (BM25 + embeddings + RRF)**	**Equilíbrio entre precisão e recall**
Indexação e busca	**PostgreSQL com pg_trgm e pgvector**	**Um só serviço, consistência e baixo custo**
Linguagem do servidor	**Python (FastAPI)**	**Ecossistema de OCR, imagem e NLP**
Frontend	**JavaScript (React)**	**Interface web móvel e captura de imagem**
Base de dados	**PostgreSQL**	**Recursos avançados e extensões**

8. TECNOLOGIAS UTILIZADAS

Este capítulo descreve a arquitetura do sistema e cada uma das tecnologias selecionadas, o papel que desempenham e as razões para a sua utilização.

8.1 Arquitetura geral

O sistema segue uma arquitetura em camadas, cliente-servidor, com comunicação por API REST. A Tabela 7 resume as camadas.

Tabela 6. Camadas da arquitetura

Camada	Responsabilidade	Tecnologias
Apresentação	Formulários, captura por câmara, resultados, painel de gestão	HTML5, CSS3, JavaScript, React (aplicação web progressiva)
Aplicação (API)	Autenticação, regras de negócio, orquestração do pipeline	Python 3, FastAPI, Pydantic, Uvicorn
Processamento de imagem e OCR	Correção, binarização, OCR, confiança por palavra	OpenCV, Pillow, Tesseract, PaddleOCR
NLP e busca	Normalização, extração de campos, embeddings, fusão de resultados	Expressões regulares, sentence-transformers, RRF
Dados	Metadados, texto, índices, vetores	PostgreSQL, pg_trgm, pgvector
Armazenamento de ficheiros	Imagens originais e miniaturas cifradas	Armazenamento compatível com S3 (MinIO) ou sistema de ficheiros
Implantação	Empacotamento, proxy reverso, TLS	Docker, Docker Compose, Nginx, Let's Encrypt

8.2 Frontend: HTML5, CSS3, JavaScript (React)

A interface é desenvolvida em **HTML5**, **CSS3** e **JavaScript**, com a biblioteca **React** para componentes reutilizáveis e gestão de estado. A captura de imagem usa a API *MediaDevices* do navegador ou o elemento *input*, o que dispensa instalar uma aplicação nativa. A interface é construída com a abordagem *mobile-first* (que foca em formatações que favorecem a vista mobile), com componentes leves, imagens comprimidas no cliente antes do envio e carregamento progressivo, de modo a funcionar com ligações lentas.

8.3 Backend: Python e FastAPI

O servidor usa **Python 3** com o *framework* **FastAPI**, executado pelo servidor ASGI **Uvicorn**. O FastAPI valida automaticamente os dados de entrada através de modelos **Pydantic**, gera documentação OpenAPI e suporta funções assíncronas, o que permite atender vários pedidos enquanto o OCR corre noutro processo. As operações mais pesadas, como OCR e geração de *embeddings*, são delegadas a uma fila de tarefas em segundo plano. O utilizador recebe uma resposta imediata e consulta o estado do processamento. Isto evita bloquear o servidor e permite escalar os trabalhadores de forma independente.

A API segue o estilo REST, com recursos como *documentos*, *buscas* e *utilizadores*. Os principais *endpoints* são: criação de registo de documento perdido ou achado; envio de imagem; consulta de estado do processamento; busca; e gestão de contactos e de reivindicações. A autenticação usa **JSON Web Tokens** (JWT) com expiração curta. As palavras-passe são guardadas com uma função de derivação própria para o efeito, como o Argon2 ou o bcrypt.

8.4 Pré-processamento de imagem: OpenCV e Pillow

A qualidade do OCR depende diretamente da qualidade da imagem. A biblioteca **OpenCV** (Bradski, 2000) fornece as operações necessárias, e o **Pillow** trata da leitura, conversão de formatos e redimensionamento. O pipeline aplica, por ordem:

- **Correção de orientação.** Rotação automática com base nos metadados EXIF e na deteção de inclinação (*deskew*).
- **Deteção do contorno do documento e correção de perspetiva.** O documento é recortado do fundo e «endireitado» por transformação de perspetiva.
- **Conversão para tons de cinzento e redução de ruído.** Filtros de suavização, como o filtro bilateral ou a média não local, preservam as arestas das letras.
- **Ampliação.** Imagens com resolução baixa são ampliadas (idealmente próximo de 300 DPI de equivalente) antes do OCR.

O sistema guarda a imagem original intacta e usa a versão tratada apenas para o OCR. A medição do efeito do pré-processamento é um dos objetivos da avaliação: compara-se a taxa de erro de caracteres com e sem cada etapa.

8.5 Reconhecimento ótico de caracteres: Tesseract e PaddleOCR

O **Tesseract**, usado através da interface *pytesseract*, é configurado com o modelo de língua portuguesa e com os modos de segmentação de página adequados a documentos curtos e estruturados. O motor devolve, além do texto, uma medida de confiança por palavra. O sistema usa essa confiança de duas formas: para decidir se aciona o motor de contingência (**PaddleOCR**) e para destacar na interface os campos duvidosos, que o utilizador é convidado a rever.

Para documentos com campos em posições conhecidas, como o BI, aplicam-se regiões de interesse (*templates*) que isolam cada campo e o leem com um modo de segmentação de linha única. Isto reduz erros de segmentação. Em campos como números de documento, restringe-se o conjunto de caracteres aceites (*whitelist*), o que evita confusões entre letras e algarismos. Os resultados do OCR passam depois por uma etapa de correção, descrita na secção seguinte.

O desempenho do OCR é medido por duas métricas: a **taxa de erro de caracteres** (CER) e a **taxa de erro de palavras** (WER), ambas baseadas na distância de edição de Levenshtein (1966), calculada entre o texto reconhecido e a transcrição de referência.

8.6 Base de dados e índices: PostgreSQL, pg_trgm e pgvector

O **PostgreSQL** guarda os utilizadores, os registos de documentos, o estado dos processos de reivindicação e o registo de auditoria. A tabela principal de documentos tem, entre outros, os campos de tipo (perdido ou achado), tipo de documento, nome normalizado, número de documento, entidade emissora, local e data, texto OCR, estado do registo, uma coluna *tsvector* para texto integral e uma coluna *vector* para o *embedding*.

Os índices usados são: **GIN** sobre a coluna *tsvector* (índice invertido para busca lexical); **GIN com operadores de trigramas** sobre nome e número (busca aproximada); e **HNSW** sobre a coluna vetorial (vizinhos mais próximos aproximados) (Malkov & Yashunin, 2020; Kane, s.d.). A configuração de língua «portuguese» aplica *stemming* e remoção de palavras vazias na busca lexical (PostgreSQL Global Development Group, 2024).

A busca híbrida é feita por consultas separadas, uma lexical e uma vetorial, cujas listas de candidatos são fundidas na aplicação com RRF, onde a pontuação de cada documento é a soma de 1/(k + posição) nas listas em que aparece, com k habitualmente igual a 60 (Cormack et al., 2009). Antes da fusão, aplicam-se filtros determinísticos, como o tipo de documento, o intervalo de datas e a zona geográfica, que reduzem o espaço de busca e melhoram a precisão. Um documento do tipo «achado» só é comparado com consultas de «perdido», e vice-versa.

8.7 Segurança, privacidade e conformidade legal

O sistema trata dados pessoais, incluindo imagens de documentos de identidade. Aplicam-se os seguintes princípios e medidas:

- **Minimização.** Recolhem-se apenas os campos necessários para a correspondência. Em registos de documentos achados, o número completo do documento nunca é mostrado publicamente; mostram-se apenas os primeiros e os últimos caracteres e o nome parcialmente ocultado.
- **Consentimento e finalidade.** O utilizador aceita explicitamente os termos de tratamento, e os dados são usados apenas para reunir documentos e titulares.
- **Cifragem.** O tráfego usa HTTPS com TLS, via Nginx e certificados Let's Encrypt. Os dados sensíveis e as imagens são cifrados em repouso.
- **Verificação de reivindicação.** Para entregar um documento, o sistema exige que o reclamante prove conhecimento de dados que não são exibidos (por exemplo, a data de nascimento) e que o contacto seja intermediado pela plataforma ou por uma instituição.

8.8 Contentorização e implantação: Docker, Nginx e CI

A aplicação é empacotada com **Docker** (Merkel, 2014), com um contentor para cada serviço: API, trabalhador de processamento, PostgreSQL, armazenamento de objetos e proxy reverso. O **Docker Compose** descreve e arranca o conjunto com um único comando, o que torna o ambiente de desenvolvimento igual ao de produção e facilita a instalação em servidores de instituições. O **Nginx** atua como proxy reverso, termina o TLS, serve os ficheiros estáticos do *frontend* e limita o ritmo de pedidos.

O código é versionado com **Git**. Uma rotina de integração contínua executa os testes automáticos e a verificação de estilo em cada alteração. O sistema pode ser alojado num servidor privado virtual de baixo custo. Para o piloto, a escolha entre alojamento em Angola ou no estrangeiro deve ponderar a latência, o custo e as obrigações de localização e de transferência de dados.

8.9 Limitações e trabalho futuro

Este trabalho tem limitações que devem ser claras. A primeira é a dependência da qualidade da imagem: mesmo com pré-processamento, documentos muito danificados podem não ser reconhecidos. A segunda é a ausência, na fase inicial, de dados reais, pelo que as métricas obtidas em dados sintéticos devem ser interpretadas com cautela. A terceira é a necessidade de articulação institucional (esquadras, serviços de identificação e administrações) sem a qual a base terá poucos registos de documentos achados.

9. CONCLUSÃO

Este trabalho propôs e fundamentou um **sistema web de recuperação de documentos perdidos**, tendo como alvo Angola. O sistema combina digitalização ou inserção textual, OCR e recuperação de informação lexical e semântica. A análise do contexto mostrou que o problema tem relevância prática: os documentos de identificação são indispensáveis para aceder a serviços e que a informação sobre documentos perdidos e achados está dispersa.

O estudo comparativo permitiu responder às questões de pesquisa no plano teórico e de desenho. Para o OCR, o Tesseract, apoiado por pré-processamento com OpenCV, oferece o melhor equilíbrio entre exatidão, custo e privacidade. Para a busca, a literatura indica que o BM25 é mais preciso em identificadores. A pilha selecionada, com Python e FastAPI, React e PostgreSQL, concentra estas capacidades num conjunto de código aberto e de baixo custo operacional.

Teoricamente, o trabalho cumpre o objetivo geral ao fundamentar o desenho técnico do sistema. As hipóteses e as respostas baseiam-se na literatura e ainda não foram validadas com testes. As principais limitações são a dependência da qualidade da imagem e a ausência de dados reais. A proteção de dados pessoais é um requisito obrigatório. Prevê-se, como próximos passos, a implementação de um protótipo funcional, o estudo de técnicas híbridas mais avançadas e um projeto-piloto

10. REFERÊNCIAS BIBLIOGRAFICAS

Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*, *25*(11), 120–125.

Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 758–759). ACM.

Du, Y., Li, C., Guo, R., Yin, X., Liu, W., Zhou, J., Bai, Y., Yu, Z., Yang, Y., Dang, Q., & Wang, H. (2020). *PP-OCR: A practical ultra lightweight OCR system* [Preprint]. arXiv. https://arxiv.org/abs/2009.09941

Gormley, C., & Tong, Z. (2015). *Elasticsearch: The definitive guide*. O'Reilly Media.

Kane, A. (s.d.). *pgvector: Open-source vector similarity search for Postgres* [Software]. GitHub. https://github.com/pgvector/pgvector

Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, *10*(8), 707–710.

Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., Zhang, C., Li, Z., & Wei, F. (2023). TrOCR: Transformer-based optical character recognition with pre-trained models. *Proceedings of the AAAI Conference on Artificial Intelligence*, *37*(11), 13094–13102.

Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *42*(4), 824–836.

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, *2014*(239), Article 2.

PostgreSQL Global Development Group. (2024). *PostgreSQL documentation*. https://www.postgresql.org/docs/